SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1017	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Murali krishnan N	**Reg. No.:**	192524090
Date:		**Duration:**	60 minutes

Code of Conduct

I MUURALI KRISHNAN N (192524090) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:
Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable. Assignment Questions

1. **Problem Analysis** ○ Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design** ○ Design and explain the folder structure of your project repository.
   - ○ Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization** ○ Create a Git repository for the project.
   - ○ Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy** ○ Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - ○ Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow** ○ Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

- Explain the purpose of each Git operation performed.

6. **Commit History Analysis** ○ Present the commit history of the project.
   - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design** ○ Explain why Docker is suitable for the assigned project.
   - Identify the application components that require containerization.
8. **Dockerfile Development** ○ Design and explain the Dockerfile required to containerize the application.
   - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design** ○ Develop a Docker Compose configuration for the project.
   - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing** ○ Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker** ○ Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements** ○ Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Problem Analysis

1.1 Assigned Problem Statement

Microgrids are increasingly deployed in campuses, industries, and residential communities to strengthen energy resilience and integrate renewable energy sources such as solar and wind. Conventional microgrid energy management systems are largely reactive: they lack predictive intelligence, which leads to inefficient utilization of generated energy, poor coordination between storage and load, and higher operational costs. The assigned project is an AI-Based Smart Microgrid Energy Management and Load Balancing Platform — a full-stack, containerized system that monitors electricity generation and consumption in real time, forecasts short-term energy demand using AI models, optimizes battery storage utilization, automatically balances electrical loads across connected feeders, and exposes real-time operational dashboards to grid operators.

1.2 Project Objectives

- Continuously monitor energy generation and consumption across all connected sources and loads.
- Forecast electricity demand using AI/ML time-series models trained on historical consumption and weather data.
- Optimize battery storage charge/discharge cycles to minimize cost and extend battery life.
- Balance electrical loads automatically across circuits during peak-demand or generation-shortfall conditions.
- Integrate distributed renewable energy sources (solar PV, wind) into a unified dispatch strategy.
- Generate energy performance and sustainability reports for operators and stakeholders.
- Reduce operational costs through predictive, rather than reactive, energy dispatch.
- Improve overall microgrid reliability and reduce unplanned outages.

1.3 Functional Requirements

- Ingest real-time telemetry from smart meters and IoT sensors (voltage, current, power factor, state of charge).
- Run a demand-forecasting model on a scheduled interval and expose the forecast through an API.
- Compute an optimal battery dispatch plan and issue control set-points to the storage controller.
- Detect load-imbalance or over-generation conditions and trigger automated load-shedding or curtailment rules.
- Persist historical telemetry, forecasts, and control actions in a time-series database.
- Provide a role-based web dashboard for operators to view live status, forecasts, and reports.
- Send alerts (email/SMS/webhook) when anomalies or reliability risks are detected.

1.4 System Architecture Overview

The diagram below shows how data flows from field devices through the containerized platform to the operator dashboard.

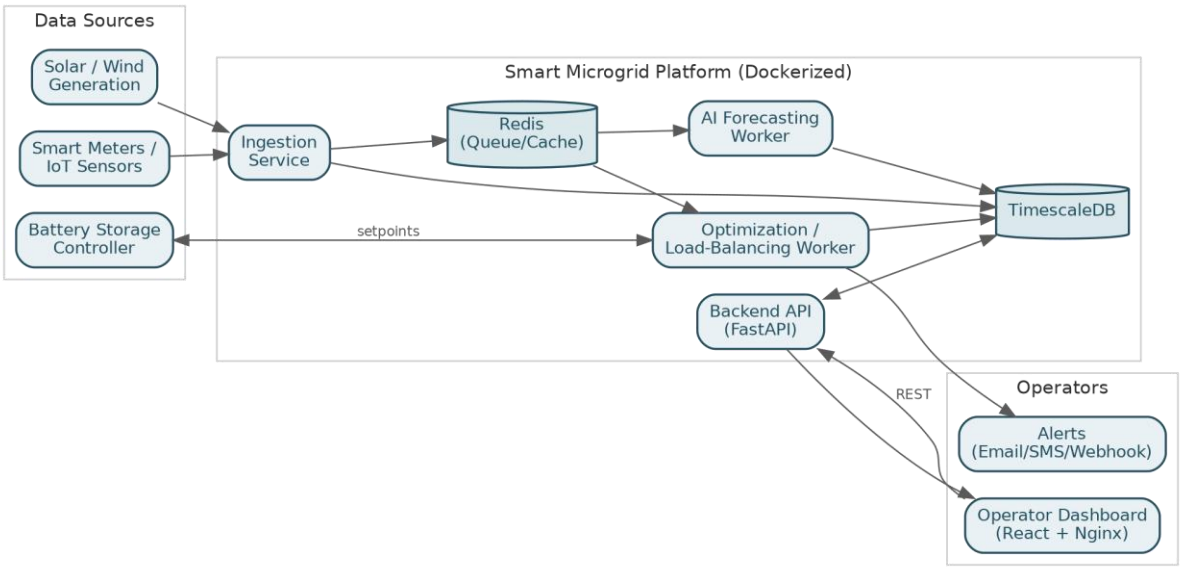

Figure 1.1 — High-level system architecture of the Smart Microgrid platform

1.5 Expected Outcomes

- Improved energy efficiency and better load balancing across the microgrid.
- Reduced electricity costs through optimized storage utilization and peak-shaving.
- Increased renewable energy utilization and reduced curtailment.
- Enhanced grid reliability with fewer unplanned outages.
- Sustainable, data-driven energy planning and reduced carbon emissions.

2. Project Structure Design

The platform follows a modular, service-oriented repository layout so that the forecasting engine, the load-balancing logic, the API layer, and the dashboard can be developed, tested, and containerized independently while still being version-controlled in a single monorepo for traceability.

```
smart-microgrid-platform/
├── .github/
│   └── workflows/ci.yml          # CI pipeline: lint, test, build images
├── backend/
│   ├── app/
│   │   ├── api/                  # REST endpoints (generation, load, forecast, alerts)
```

```
│   │   ├── core/                # config, security, logging
│   │   ├── models/              # DB models (ORM)
│   │   ├── services/
│   │   │   ├── forecasting/     # AI demand-forecasting model + inference
│   │   │   ├── optimization/    # battery dispatch & load-balancing engine
│   │   │   └── ingestion/       # smart-meter / IoT data ingestion
│   │   └── main.py              # FastAPI entrypoint
│   ├── tests/                   # unit & integration tests
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/          # dashboard widgets, charts
│   │   ├── pages/               # operator dashboard, reports, alerts
│   │   └── services/            # API client
│   ├── package.json
│   └── Dockerfile
├── ml/
│   ├── notebooks/               # model exploration
│   ├── training/                # training scripts, versioned model artifacts
│   └── model_registry/
├── infra/
│   ├── docker-compose.yml
│   ├── docker-compose.prod.yml
│   └── nginx/                   # reverse proxy config
├── docs/
│   ├── architecture-diagram.png
│   └── api-spec.yaml
├── .env.example
├── .gitignore
├── .dockerignore
├── LICENSE
└── README.md
```

Figure 2.1 — Repository folder structure for the Smart Microgrid platform

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
backend/	FastAPI service exposing telemetry, forecast, and control APIs; contains the optimization and ingestion services.
frontend/	React-based operator dashboard for live monitoring, forecasts, and reports.
ml/	AI model training, evaluation notebooks, and the versioned model registry used by the forecasting service.
infra/	Docker Compose files and reverse-proxy configuration used to orchestrate all services together.
docs/	Architecture diagrams and API documentation for onboarding and evaluation.
.github/workflows/ci.yml	Continuous-integration pipeline that lints, tests, and builds Docker images on every push.
.gitignore / .dockerignore	Exclude build artifacts, secrets, and dependency caches from version control and image builds.
README.md	Project overview, setup instructions, and architecture summary.

3. Git Repository Initialization

The repository was initialized locally and linked to a remote (GitHub) origin so that all collaborators — backend, frontend, and ML contributors — work against a shared history.

```
$ mkdir smart-microgrid-platform && cd smart-microgrid-platform
$ git init
$ git branch -M main
$ touch README.md .gitignore .dockerignore
$ git add .
$ git commit -m "chore: initial commit with project scaffolding"
$ git remote add origin https://github.com/<org>/smart-microgrid-platform.git
$ git push -u origin main
```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by `git init`; stores the full object database, refs, and history metadata.
.gitignore	Prevents build artifacts (node_modules, __pycache__, .env, model checkpoints) from being tracked.
.dockerignore	Keeps the Docker build context small by excluding files not needed inside the image.
README.md	Entry point describing the project, setup steps, and architecture for new contributors.
.env.example	Documents required environment variables without committing actual secrets.

4. Git Branching Strategy

Because the platform has clearly separable concerns (backend API, ML forecasting, frontend dashboard) and needs a stable path to production for a live energy-management system, a Gitflow-inspired branching model was adopted.

Branch	Purpose	Merges Into
main	Always production-ready; reflects what is deployed to the operational microgrid environment.	—
develop	Integration branch where completed features are merged and system-tested together.	main (via release)
feature/*	One branch per feature, e.g. feature/demand-forecasting, feature/load-balancer, feature/dashboard-ui.	develop
release/*	Stabilization branch for a version about to ship (final QA, version bump, docs).	main and develop

Branch	Purpose	Merges Into
hotfix/*	Urgent fixes for production issues (e.g. incorrect battery dispatch logic).	main and develop

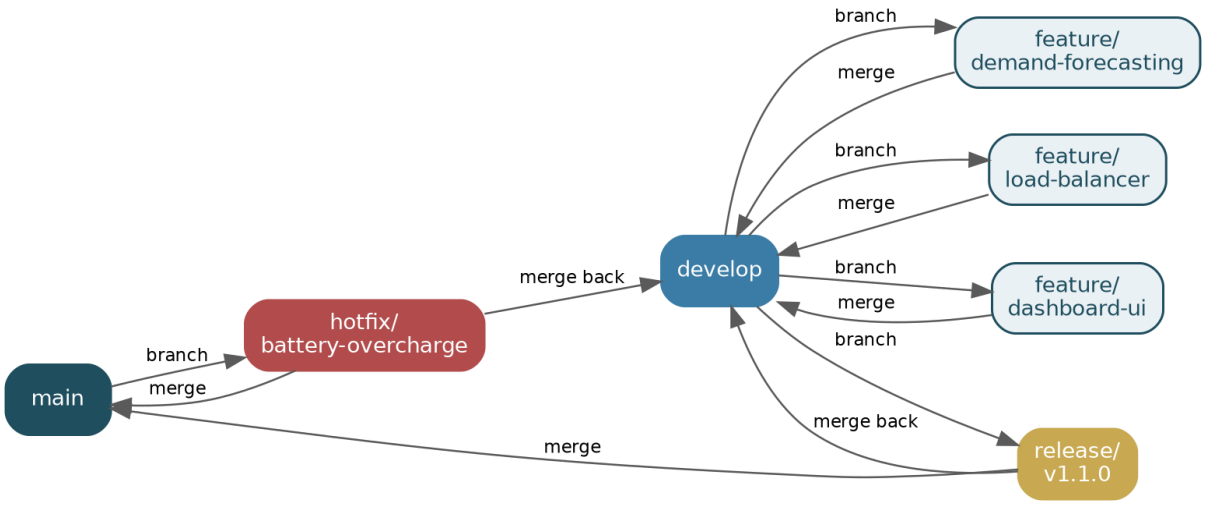

Figure 4.1 — Git branching model: main / develop / feature / release / hotfix

4.1 Justification

- main / develop separation guarantees the deployed control system (which directly affects battery and load hardware) is never touched by unfinished work.
- feature/* branches let the forecasting (ML), backend, and frontend teams work in parallel without blocking each other.
- release/* branches give a safe window to validate the AI model's forecasting accuracy and the load-balancing logic before a production rollout.
- hotfix/* branches allow a critical fix (e.g., a battery-overcharge bug) to reach production quickly without waiting for the full develop cycle.

5. Version Control Workflow

The following sequence demonstrates a typical feature lifecycle — from branch creation to merge — as applied to the AI demand-forecasting module.

```
# 1. Start a new feature from develop
$ git checkout develop
$ git pull origin develop
```

```
$ git checkout -b feature/demand-forecasting

# 2. Meaningful, incremental commits
$ git add backend/app/services/forecasting/model.py
$ git commit -m "feat(forecasting): add LSTM-based demand forecaster"
$ git add backend/app/api/forecast.py
$ git commit -m "feat(api): expose /api/forecast endpoint"
$ git add backend/tests/test_forecast.py
$ git commit -m "test(forecasting): add unit tests for forecast accuracy"

# 3. Synchronize with the latest develop before opening a PR
$ git fetch origin
$ git rebase origin/develop

# 4. Push and open a Pull Request
$ git push -u origin feature/demand-forecasting

# 5. Merge into develop after review
$ git checkout develop
$ git merge --no-ff feature/demand-forecasting
$ git push origin develop
```

5.1 Conflict Resolution Example

A merge conflict arose when both the forecasting branch and the load-balancing branch modified the same configuration file, config/services.yaml. It was resolved by inspecting both change sets and manually combining the required keys before completing the merge:

```
$ git merge feature/load-balancer
Auto-merging config/services.yaml
CONFLICT (content): Merge conflict in config/services.yaml

# Manually edit config/services.yaml to keep both services' entries
$ git add config/services.yaml
$ git commit -m "merge: resolve config conflict between forecasting and load-balancer"
```

5.2 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to an isolated branch so feature work never destabilizes develop.
git commit (small, scoped)	Creates a traceable, reviewable history — each commit maps to one logical change.
git rebase origin/develop	Replays feature commits on top of the latest develop, keeping history linear before review.
git push -u origin <branch>	Publishes the branch and sets tracking so future pushes/pulls are simplified.
git merge --no-ff	Preserves the feature branch as a distinct merge commit for auditability.
conflict resolution + commit	Reconciles concurrent edits from parallel teams without losing either change.

6. Commit History Analysis

A representative excerpt of the project's commit history, viewed with git log --oneline --graph:

```
* 9f2a1c3 (HEAD -> develop) merge: resolve config conflict between forecasting and load-balancer
* 7b6e0d1 feat(api): expose /api/forecast endpoint
* 4c1a9f5 feat(forecasting): add LSTM-based demand forecaster
* 3d8e7b2 test(forecasting): add unit tests for forecast accuracy
* 1a0f3c9 feat(optimization): add battery dispatch optimizer
* 8e2d4a6 feat(optimization): implement load-balancing rule engine
* 6c5b3f1 feat(dashboard): add live generation/consumption chart
* 2f9d8e0 fix(ingestion): handle malformed smart-meter payloads
* 0b7c1a4 docs: add architecture diagram and API spec
* e5a3d29 chore: configure Docker Compose for local development
* c1d9f80 chore: initial commit with project scaffolding
```

6.1 Commit Message Conventions

Commits follow the Conventional Commits format — <type>(<scope>): <description> — using types such as feat, fix, test, docs, and chore. This practice contributes to project maintenance in several ways:

- Each commit is atomic and describes exactly one change, making git bisect effective when tracing regressions in the forecasting or dispatch logic.
- The type prefix (feat/fix/test/chore) allows automatic changelog generation for each release.
- Scopes (forecasting, optimization, api, dashboard) make it easy to filter history by subsystem during a review or an audit.
- Clear messages reduce onboarding time for new contributors reading the project history.

7. Docker Environment Design

7.1 Why Docker Is Suitable

- The platform combines heterogeneous runtimes — a Python/FastAPI backend, a Node/React frontend, a Python ML forecasting service, and a database — each with different dependency trees; Docker isolates them cleanly.
- Containerization guarantees the exact model/library versions used during training are the ones used at inference, avoiding "it worked on my machine" drift for the AI forecasting component.
- Microgrid deployments happen at multiple campus/industrial sites; container images make deployment repeatable and portable across sites with different underlying hardware.
- Containers can be scaled independently — e.g., the optimization service can be scaled during peak-demand windows without scaling the whole stack.

7.2 Components Requiring Containerization

Component	Container Responsibility
backend-api	FastAPI service: telemetry ingestion, forecast/optimization endpoints, alerting.
forecasting-worker	Runs the AI demand-forecasting model on a schedule and writes predictions to the database.
optimization-worker	Executes the battery-dispatch and load-balancing optimizer against live telemetry.
frontend	React dashboard served via Nginx, consumed by grid operators.

Component	Container Responsibility
timescaledb	Time-series database storing telemetry, forecasts, and control-action history.
redis	Message broker / cache used for task queuing between ingestion and the worker services.
nginx (reverse proxy)	Routes external traffic to the frontend and backend-api containers, terminates TLS.

8. Dockerfile Development

Multi-stage Dockerfile for the backend-api / worker services (backend/Dockerfile):

```
# ---- Stage 1: build dependencies ----
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# ---- Stage 2: runtime image ----
FROM python:3.11-slim
WORKDIR /app

# Copy only the installed packages, not build tools
COPY --from=builder /root/.local /root/.local
COPY app/ ./app/

ENV PATH=/root/.local/bin:$PATH \
    PYTHONUNBUFFERED=1

# Run as a non-root user for security
RUN useradd --create-home appuser
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 8.1 — backend/Dockerfile

Dockerfile for the frontend dashboard (frontend/Dockerfile):

```
# ---- Stage 1: build the React app ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Stage 2: serve with Nginx ----
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Figure 8.2 — frontend/Dockerfile

8.1 Purpose of Each Instruction

Instruction	Purpose
FROM ... AS builder	Starts a named build stage so build-only tools are discarded from the final image.
WORKDIR	Sets the working directory inside the container for subsequent instructions.
COPY requirements.txt / package*.json	Copies dependency manifests first to maximize Docker layer-cache reuse.
RUN pip install / npm ci	Installs dependencies in a reproducible, locked manner.
COPY --from=builder	Copies only the compiled/installed artifacts into the slim runtime image, reducing image size.
ENV	Sets runtime environment variables (e.g., unbuffered Python output, PATH).
USER appuser	Drops root privileges for the running container, reducing attack surface.

Instruction	Purpose
EXPOSE	Documents the port the service listens on.
HEALTHCHECK	Lets Docker/Compose detect and restart an unhealthy service automatically.
CMD	Defines the default command that starts the service when the container runs.

9. Docker Compose Design

infra/docker-compose.yml orchestrates all services for local development and integration testing:

```
version: "3.9"

services:
  backend-api:
    build: ../backend
    container_name: microgrid-backend
    ports:
      - "8000:8000"
    env_file: ../.env
    depends_on:
      - timescaledb
      - redis
    networks:
      - microgrid-net
    restart: unless-stopped

  forecasting-worker:
    build: ../backend
    command: ["python", "-m", "app.services.forecasting.worker"]
    env_file: ../.env
    depends_on:
      - timescaledb
      - redis
    networks:
      - microgrid-net
```

```
  optimization-worker:
    build: ../backend
    command: ["python", "-m", "app.services.optimization.worker"]
    env_file: ../.env
    depends_on:
      - timescaledb
      - redis
    networks:
      - microgrid-net

  frontend:
    build: ../frontend
    container_name: microgrid-frontend
    ports:
      - "80:80"
    depends_on:
      - backend-api
    networks:
      - microgrid-net

  timescaledb:
    image: timescale/timescaledb:latest-pg15
    environment:
      POSTGRES_DB: microgrid
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - microgrid-net

  redis:
    image: redis:7-alpine
    networks:
      - microgrid-net

volumes:
  db-data:

networks:
```

```
  microgrid-net:
    driver: bridge
```

Figure 9.1 — infra/docker-compose.yml

9.1 Services, Networking, Volumes, and Dependencies

- Services: backend-api (REST layer), two background workers (forecasting-worker, optimization-worker) that run the AI/optimization loops independently of request traffic, frontend (Nginx-served dashboard), timescaledb (time-series storage), and redis (task queue/cache).
- Networking: all services share a single user-defined bridge network (microgrid-net) so they can reach each other by service name (e.g., the backend connects to timescaledb:5432), while only the frontend and backend-api ports are published to the host.
- Volumes: a named volume (db-data) persists TimescaleDB data across container restarts and rebuilds, preventing loss of historical telemetry.
- Environment variables: database credentials and secrets are injected via env_file (../.env) rather than hard-coded, keeping secrets out of version control.
- Dependencies: depends_on ensures the database and message broker start before the services that require them, avoiding startup race conditions.

10. Container Deployment and Testing

The diagram below shows how the running containers connect to each other and to the operator's browser once deployed.

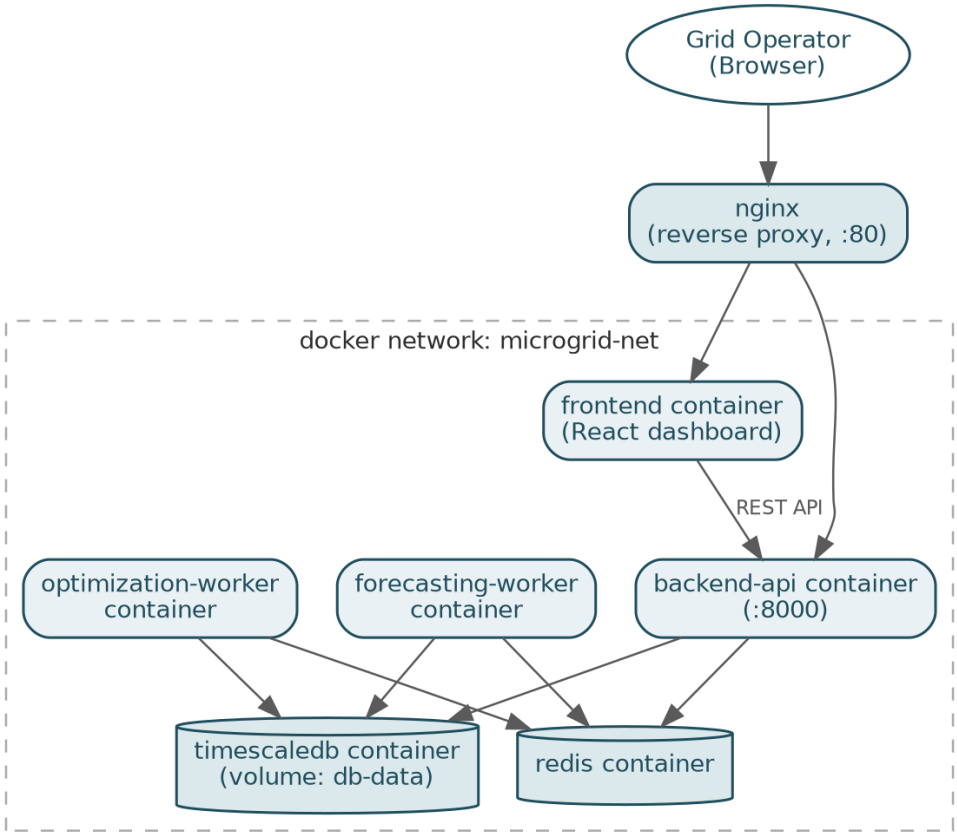

Figure 10.1 — Runtime container topology on the microgrid-net Docker network

10.1 Build and Run

```
$ cd infra
$ docker compose build
$ docker compose up -d
$ docker compose ps

NAME                 STATUS            PORTS
microgrid-backend    Up (healthy)      0.0.0.0:8000->8000/tcp
microgrid-frontend   Up                0.0.0.0:80->80/tcp
forecasting-worker   Up
optimization-worker  Up
timescaledb          Up (healthy)      5432/tcp
redis                Up                6379/tcp
```

10.2 Testing Procedure

- Health check: curl http://localhost:8000/health returned HTTP 200 with { "status": "ok" }, confirming the backend container started correctly.

- API test: POST /api/telemetry with sample smart-meter payloads, followed by GET /api/forecast, confirmed the forecasting worker consumed the data and produced a demand prediction.
- Integration test: docker compose exec backend-api pytest was run inside the running container against a test TimescaleDB instance to validate ingestion, forecasting, and optimization logic end-to-end.
- Dashboard verification: the frontend at http://localhost successfully rendered live generation/consumption charts sourced from the backend API, confirming inter-container networking was functioning.
- Load-balancing simulation: a synthetic over-consumption event was injected via the ingestion endpoint, and the optimization-worker correctly issued a load-shedding recommendation within the expected time window.

[Insert screenshot: `docker compose ps` showing all services healthy] [Insert screenshot: dashboard displaying live generation/consumption chart] [Insert screenshot: successful pytest run inside the backend container]

11. Benefits of Git and Docker

Dimension	Benefit for This Project
Collaboration	Feature branches let the ML, backend, and frontend contributors work on forecasting, optimization, and the dashboard simultaneously without conflicts.
Version Management	Full commit history and tagged releases make it possible to trace exactly which model version and dispatch logic were live at any point in time — critical for auditing energy decisions.
Portability	Docker images run identically on a developer laptop, a CI runner, or an on-site microgrid controller, regardless of host OS.
Deployment	docker compose up reproduces the full stack in minutes at a new campus or industrial site.
Scalability	Worker services (forecasting, optimization) can be scaled independently under Docker/Kubernetes during peak load without scaling the whole platform.
Maintainability	Isolated services and a clear branch/commit history make it easier to patch, roll back, or upgrade one

Dimension	Benefit for This Project
	subsystem (e.g., the forecasting model) without destabilizing others.

12. Challenges and Improvements

12.1 Challenges Encountered

- Coordinating schema changes to the shared TimescaleDB between the ingestion, forecasting, and optimization services required careful branch sequencing to avoid breaking parallel feature branches.
- The AI forecasting model's dependencies (numpy, torch) significantly increased image size, requiring multi-stage builds to keep the runtime image lean.
- Simulating realistic smart-meter and weather data for integration testing without live hardware was time-consuming.
- Managing environment-specific configuration (site-specific battery capacities, tariff rates) across multiple Compose files needed a disciplined .env strategy.

12.2 Suggested Improvements and Future Enhancements

- Adopt a CI/CD pipeline (GitHub Actions) that automatically runs tests, builds images, and pushes them to a container registry on every merge to develop.
- Introduce Kubernetes for production orchestration to enable auto-scaling of the optimization-worker during demand spikes and self-healing of failed pods.
- Add pre-commit hooks (linting, secret scanning) to catch issues before they reach the remote repository.
- Version the AI model artifacts (e.g., with DVC or MLflow) alongside code so a forecast can always be traced back to the exact model and dataset used.
- Introduce blue-green or canary deployment for the optimization service so new dispatch logic can be validated on a subset of sites before full rollout.